

Chapter 2. Passing code with behavior parameterization

This chapter covers

- Coping with changing requirements
- Behavior parameterization
- Anonymous classes
- Preview of lambda expressions
- Real-world examples: Comparator, Runnable, and GUI

A well-known problem in software engineering is that no matter what you do, user requirements will change. For example, imagine an application to help a farmer understand his inventory. The farmer might want a functionality to find all green apples in his inventory. But the next day he might tell you, “Actually, I also want to find all apples heavier than 150 g.” Two days later, the farmer comes back and adds, “It would be really nice if I could find all apples that are green *and* heavier than 150 g.” How can you cope with these changing requirements? Ideally, you’d like to minimize your engineering effort. In addition, similar new functionalities ought to be straightforward to implement and maintainable in the long term.

Behavior parameterization is a software development pattern that lets you handle frequent requirement changes. In a nutshell, it means taking a block of code and making it available without executing it. This block of code can be called later by other parts of your programs, which means that you can defer the execution of that block of code. For instance, you could pass the block of code as an argument to another method that will execute it later. As a result, the method’s behavior is parameterized based on that block of code. For example, if you process a collection, you may want to write a method that

- Can do “something” for every element of a list
- When you finish processing the list
- If you encounter an error

Other formats available



This is what *behavior parameterization* refers to. Here’s an analogy: your roommate knows how to drive to the supermarket and back home. You

equivalent to calling a method *generally* passing a list of products as its

argument. But one day you're at the office, and you need him to do something he's never done before—pick up a package from the post office. You need to pass him a list of instructions: go to the post office, use this reference number, talk to the manager, and pick up the parcel. You could pass him the list of instructions by email, and when he receives it, he can follow the instructions. You've now done something a bit more advanced that's equivalent to a method `goAndDo`, which can execute various new behaviors as arguments.

We'll start this chapter by walking you through an example of how you can evolve your code to be more flexible for changing requirements. Building on this knowledge, we show how to use behavior parameterization for several real-world examples. For example, you may have used the behavior parameterization pattern already, using existing classes and interfaces in the Java API to sort a `List`, to filter names of files, or to tell a `Thread` to execute a block of code or even perform GUI event handling. You'll soon realize that this pattern is historically verbose in Java. Lambda expressions in Java 8 onward tackle the problem of verbosity. We'll show in [chapter 3](#) how to construct lambda expressions, where to use them, and how you can make your code more concise by adopting them.

2.1. Coping with changing requirements

Writing code that can cope with changing requirements is difficult. Let's walk through an example that we'll gradually improve, showing some best practices for making your code more flexible. In the context of a farm-inventory application, you have to implement a functionality to filter *green* apples from a list. Sounds easy, right?

2.1.1. First attempt: filtering green apples

Assume, as in [chapter 1](#), you have a `Color` enum available to represent different colors of an apple:

```
enum Color { RED, GREEN }
```

A first solution might be as follows:

Other formats available



Audiobook



```
public List<Apple> filterGreenApples(List<Apple> inventory) {  
    List<Apple> result = new ArrayList<>();  
    for (Apple apple : inventory) {  
        if (Color.GREEN.equals(apple.getColor()) ) {  
            result.add(apple);  
        }  
    }  
}
```

1
2

```
        return result;
    }
}
```

- **1 An accumulator list for apples**
- **2 Selects only green apples**

The highlighted line shows the condition required to select green apples. You can assume that you have a `Color` enum with a set of colors, such as `GREEN`, available. But now the farmer changes his mind and wants to also filter *red* apples. What can you do? A naïve solution would be to duplicate your method, rename it as `filterRedApples`, and change the `if` condition to match red apples. However, this approach doesn't cope well with changes if the farmer wants multiple colors. A good principle is this: when you find yourself writing nearly repeated code, try to abstract instead.

2.1.2. Second attempt: parameterizing the color

How do we avoid duplicating most of the code in `filterGreenApples` to make `filterRedApples`? To parameterize the color and be more flexible to such changes, what you could do is add a parameter to your method:

```
public static List<Apple> filterApplesByColor(List<Apple> inventory,
Color color) {
    List<Apple> result = new ArrayList<>();
    for (Apple apple: inventory) {
        if ( apple.getColor().equals(color) ) {
            result.add(apple);
        }
    }
    return result;
}
```

You can now make the farmer happy and invoke your method as follows:

```
List<Apple> greenApples = filterApplesByColor(inventory, GREEN);
List<Apple> redApples = filterApplesByColor(inventory, RED);
...
```

Other formats available



... the example a bit. The farmer comes
... be really cool to differentiate between
light apples and heavy apples. Heavy apples typically have a weight
greater than 150 g.”

Wearing your software engineering hat, you realize in advance that the farmer may want to vary the weight. So you create the following method to cope with various weights through an additional parameter:

```
public static List<Apple> filterApplesByWeight(List<Apple> inventory,
int weight) {
    List<Apple> result = new ArrayList<>();
    For (Apple apple: inventory){
        if ( apple.getWeight() > weight ) {
            result.add(apple);
        }
    }
    return result;
}
```

This is a good solution, but notice how you have to duplicate most of the implementation for traversing the inventory and applying the filtering criteria on each apple. This is somewhat disappointing because it breaks the DRY (don't repeat yourself) principle of software engineering. What if you want to alter the filter traversing to enhance performance? You now have to modify the implementation of *all* of your methods instead of only a single one. This is expensive from an engineering-effort perspective.

You could combine the color and weight into one method, called `filter`. But then you'd still need a way to differentiate what attribute you want to filter on. You could add a flag to differentiate between color and weight queries. (But never do this! We'll explain why shortly.)

2.1.3. Third attempt: filtering with every attribute you can think of

An ugly attempt to merge all attributes might be as follows:

```
public static List<Apple> filterApples(List<Apple> inventory, Color co
int weight, boolean flag) {
    List<Apple> result = new ArrayList<>();
    for (Apple apple: inventory) {
        if ( (flag && apple.getColor().equals(color)) ||
            (!flag && apple.getWeight() > weight) ){
            result.add(apple);
        }
    }
}
```

Other formats available



- 1 An ugly way to select color or weight

You could use this as follows (but it's ugly):

```
List<Apple> greenApples = filterApples(inventory, GREEN, 0, true);
List<Apple> heavyApples = filterApples(inventory, null, 150, false);
...
```

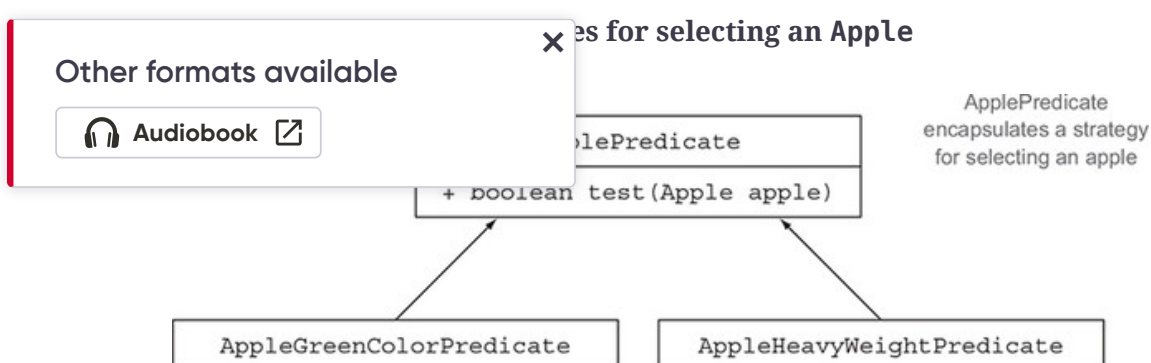
This solution is extremely bad. First, the client code looks terrible. What do `true` and `false` mean? In addition, this solution doesn't cope well with changing requirements. What if the farmer asks you to filter with different attributes of an apple, for example, its size, its shape, its origin, and so on? Furthermore, what if the farmer asks you for more complicated queries that combine attributes, such as green apples that are also heavy? You'd either have multiple duplicated filter methods or one hugely complex method. So far, you've parameterized the `filterApples` method *with values* such as a `String`, an `Integer`, an enum type, or a `boolean`. This can be fine for certain well-defined problems. But in this case, what you need is a better way to tell your `filterApples` method the selection criteria for apples. In the next section, we describe how to make use of *behavior parameterization* to attain that flexibility.

2.2. Behavior parameterization

You saw in the previous section that you need a better way than adding lots of parameters to cope with changing requirements. Let's step back and find a better level of abstraction. One possible solution is to model your selection criteria: you're working with apples and returning a `boolean` based on some attributes of `Apple`. For example, is it green? Is it heavier than 150 g? We call this a *predicate* (a function that returns a `boolean`). Let's therefore define an interface *to model the selection criteria*:

```
public interface ApplePredicate{
    boolean test (Apple apple);
}
```

You can now declare multiple implementations of `ApplePredicate` to represent different selection criteria, as shown in the following (and illustrated in [figure 2.1](#)):



```

public class AppleHeavyWeightPredicate implements ApplePredicate {    1
    public boolean test(Apple apple) {
        return apple.getWeight() > 150;
    }
}
public class AppleGreenColorPredicate implements ApplePredicate {    2
    public boolean test(Apple apple) {
        return GREEN.equals(apple.getColor());
    }
}

```

- 1 Selects only heavy apples
- 2 Selects only green apples

You can see these criteria as different behaviors for the `filter` method. What you just did is related to the strategy design pattern (see http://en.wikipedia.org/wiki/Strategy_pattern), which lets you define a family of algorithms, encapsulate each algorithm (called a strategy), and select an algorithm at run time. In this case the family of algorithms is `ApplePredicate` and the different strategies are `AppleHeavyWeightPredicate` and `AppleGreenColorPredicate`.

But how can you make use of the different implementations of `ApplePredicate`? You need your `filterApples` method to accept `ApplePredicate` objects to test a condition on an `Apple`. This is what *behavior parameterization* means: the ability to tell a method to *take* multiple behaviors (or strategies) as parameters and use them internally to *accomplish* different behaviors.

To achieve this in the running example, you add a parameter to the `filterApples` method to take an `ApplePredicate` object. This has a great software engineering benefit: you can now separate the logic of iterating the collection inside the `filterApples` method with the behavior you want to apply to each element of the collection (in this case a predicate).

2.2.1. Fourth attempt: filtering by abstract criteria

Our modified `filter` method, which uses an `ApplePredicate`, looks like this:

Other formats available



```

... filterApples(List<Apple> inventory,
                ApplePredicate p) {
    List<Apple> result = new ArrayList<>();
    for(Apple apple: inventory) {
        if(p.test(apple)) {
            result.add(apple);
        }
    }
}

```

```

    }
    return result;
}

```

- **1 Predicate *p* encapsulates the condition to test on an apple.**

Passing code/behavior

It's worth pausing for a moment for a small celebration. This code is much more flexible than our first attempt, but at the same time it's easy to read and to use! You can now create different `ApplePredicate` objects and pass them to the `filterApples` method. Free flexibility! For example, if the farmer asks you to find all red apples that are heavier than 150 g, all you need to do is create a class that implements the `ApplePredicate` accordingly. Your code is now flexible enough for any change of requirements involving the attributes of `Apple`:

```

public class AppleRedAndHeavyPredicate implements ApplePredicate {
    public boolean test(Apple apple){
        return RED.equals(apple.getColor())
            && apple.getWeight() > 150;
    }
}

List<Apple> redAndHeavyApples =
    filterApples(inventory, new AppleRedAndHeavyPredicate());

```

You've achieved something cool; the behavior of the `filterApples` method depends on the *code you pass* to it via the `ApplePredicate` object. You've parameterized the behavior of the `filterApples` method!

Note that in the previous example, the only code that matters is the implementation of the `test` method, as illustrated in [figure 2.2](#); this is what defines the new behaviors for the `filterApples` method. Unfortunately, because the `filterApples` method can only take objects, you have to wrap that code inside an `ApplePredicate` object. What you're doing is similar to passing code inline, because you're passing a boolean expression through an object that implements the `test` method. You'll see in [section 2.3](#) (and in more detail in [chapter 3](#)) that by using lambdas, you

Other formats available

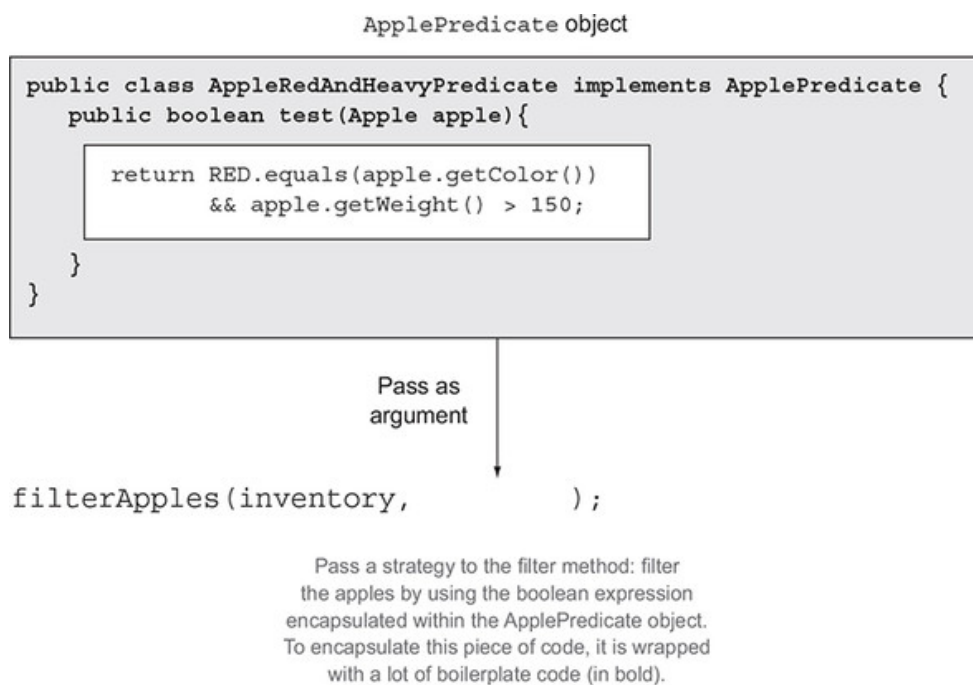


Audiobook



in `RED.equals(apple.getColor()) &&`
to the `filterApples` method without hav-
predicate classes. This removes unneces-

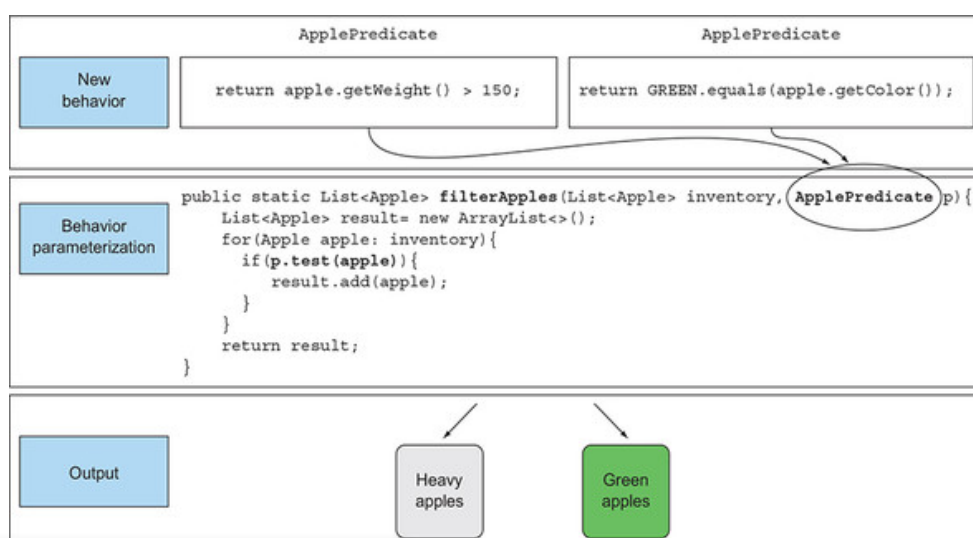
Figure 2.2. Parameterizing the behavior of `filterApples` and passing different filter strategies



Multiple behaviors, one parameter

As we explained earlier, behavior parameterization is great because it enables you to separate the logic of iterating the collection to filter and the behavior to apply on each element of that collection. As a consequence, you can reuse the same method and give it different behaviors to achieve different things, as illustrated in [figure 2.3](#). This is why *behavior parameterization* is a useful concept you should have in your toolset for creating flexible APIs.

Figure 2.3. Parameterizing the behavior of filterApples and passing different filter strategies



Other formats available



Audiobook



able with the idea of behavior parameteri-

Quiz 2.1: Write a flexible prettyPrintApple method

Write a `prettyPrintApple` method that takes a `List` of `Apples` and that can be parameterized with multiple ways to generate a `String` output from an `apple` (a bit like multiple customized `toString` methods). For example, you could tell your `prettyPrintApple` method to print only the weight of each apple. In addition, you could tell your `prettyPrintApple` method to print each apple individually and mention whether it's heavy or light. The solution is similar to the filtering examples we've explored so far. To help you get started, we provide a rough skeleton of the `prettyPrintApple` method:

```
public static void prettyPrintApple(List<Apple> inventory, ???) {
    for(Apple apple: inventory) {
        String output = ???.(apple);
        System.out.println(output);
    }
}
```

Answer:

First, you need a way to represent a behavior that takes an `Apple` and returns a formatted `String` result. You did something similar when you created an `Apple-Predicate` interface:

```
public interface AppleFormatter {
    String accept(Apple a);
}
```

You can now represent multiple formatting behaviors by implementing the `Apple-Formatter` interface:

```
public class AppleFancyFormatter implements AppleFormatter {
    public String accept(Apple apple) {
        String characteristic = apple.getWeight() > 150 ? "heavy" : "light";
        return "A " + characteristic + " " + apple.getColor() + " apple";
    }
}

public class AppleSimpleFormatter implements AppleFormatter {
```

Other formats available



Finally, you need to tell your `prettyPrintApple` method to take `AppleFormatter` objects and use them internally. You can do this by

adding a parameter to prettyPrintApple:

```
public static void prettyPrintApple(List<Apple> inventory,
                                   AppleFormatter formatter) {
    for(Apple apple: inventory) {
        String output = formatter.accept(apple);
        System.out.println(output);
    }
}
```

Bingo! You're now able to pass multiple behaviors to your prettyPrintApple method. You do this by instantiating implementations of AppleFormatter and giving them as arguments to prettyPrintApple:

```
prettyPrintApple(inventory, new AppleFancyFormatter());
```

This will produce an output along the lines of

```
A light green apple
A heavy red apple
...
```

Or try this:

```
prettyPrintApple(inventory, new AppleSimpleFormatter());
```

This will produce an output along the lines of

```
An apple of 80g
An apple of 155g
...
```

You've seen that you can abstract over behavior and make your code adapt to requirement changes, but the process is verbose because you

Other formats available



Audiobook



is that you instantiate only once. Let's see

2.3. Tackling verbosity

We all know that a feature or concept that's cumbersome to use will be avoided. At the moment, when you want to pass new behavior to your

`filterApples` method, you're forced to declare several classes that implement the `ApplePredicate` interface and then instantiate several `ApplePredicate` objects that you allocate only once, as shown in the following listing that summarizes what you've seen so far. There's a lot of verbosity involved and it's a time-consuming process!

Listing 2.1. Behavior parameterization: filtering apples with predicates

```
public class AppleHeavyWeightPredicate implements ApplePredicate {
    public boolean test(Apple apple) {
        return apple.getWeight() > 150;
    }
}

public class AppleGreenColorPredicate implements ApplePredicate {
    public boolean test(Apple apple) {
        return GREEN.equals(apple.getColor());
    }
}

public class FilteringApples {
    public static void main(String...args) {
        List<Apple> inventory = Arrays.asList(new Apple(80, GREEN),
                                              new Apple(155, GREEN),
                                              new Apple(120, RED));

        List<Apple> heavyApples =
            filterApples(inventory, new AppleHeavyWeightPredicate());
        List<Apple> greenApples =
            filterApples(inventory, new AppleGreenColorPredicate());
    }

    public static List<Apple> filterApples(List<Apple> inventory,
                                           ApplePredicate p) {
        List<Apple> result = new ArrayList<>();
        for (Apple apple : inventory) {
            if (p.test(apple)){
                result.add(apple);
            }
        }
        return result;
    }
}
```

- 1 Selects heavy apples

Other formats available



Audiobook



...ing one Apple of 155 g

...ing two green Apples

This is unnecessary overhead. Can you do better? Java has mechanisms called *anonymous classes*, which let you declare and instantiate a class at the same time. They enable you to improve your code one step further by

making it a little more concise. But they're not entirely satisfactory.

Section 2.3.3 anticipates the next chapter with a short preview of how lambda expressions can make your code more readable.

2.3.1. Anonymous classes

Anonymous classes are like the local classes (a class defined in a block) that you're already familiar with in Java. But anonymous classes don't have a name. They allow you to declare and instantiate a class at the same time. In short, they allow you to create ad hoc implementations.

2.3.2. Fifth attempt: using an anonymous class

The following code shows how to rewrite the filtering example by creating an object that implements `ApplePredicate` using an anonymous class:

```
List<Apple> redApples = filterApples(inventory, new ApplePredicate() {  
    public boolean test(Apple apple){  
        return RED.equals(apple.getColor());  
    }  
});
```

- **1 Parameterizes the behavior of the method `filterApples` with an anonymous class.**

Anonymous classes are often used in the context of GUI applications to create event-handler objects. We don't want to bring back painful memories of Swing, but the following is a common pattern that you see in practice (here using the JavaFX API, a modern UI platform for Java):

```
button.setOnAction(new EventHandler<ActionEvent>() {  
    public void handle(ActionEvent event) {  
        System.out.println("Whoooo a click!!");  
    }  
});
```

But anonymous classes are still not good enough. First, they tend to be bulky because they take a lot of space, as shown in the boldface code here

Other formats available



used previously:

```
button.setOnAction(new EventHandler<ActionEvent>() {  
    public boolean test(Apple a){  
        return RED.equals(a.getColor());  
    }  
});
```

```

public void handle(ActionEvent event) {
    System.out.println("Whoooo a click!!");
}

```

- **1 Lots of boilerplate code**

Second, many programmers find them confusing to use. For example, quiz 2.2 shows a classic Java puzzler that catches most programmers off guard! Try your hand at it.

Quiz 2.2: Anonymous class puzzler

What will the output be when this code is executed: 4, 5, 6, or 42?

```

public class MeaningOfThis {
    public final int value = 4;
    public void doIt() {
        int value = 6;
        Runnable r = new Runnable() {
            public final int value = 5;
            public void run(){
                int value = 10;
                System.out.println(this.value);
            }
        };
        r.run();
    }
    public static void main(String...args) {
        MeaningOfThis m = new MeaningOfThis();
        m.doIt();
    }
}

```

- **1 What's the output of this line?**

Answer:

The answer is 5, because this refers to the enclosing Runnable, not the enclosing class MeaningOfThis.

Other formats available



Audiobook

discourages the use of a language feature

because it takes a long time to write and maintain verbose code, and it's not pleasant to read! Good code should be easy to comprehend at a glance. Even though anonymous classes somewhat tackle the verbosity associated with declaring multiple concrete classes for an interface,

they're still unsatisfactory. In the context of passing a simple piece of code (for example, a boolean expression representing a selection criterion), you still have to create an object and explicitly implement a method to define a new behavior (for example, the method `test` for `Predicate` or the method `handle` for `EventHandler`).

Ideally we'd like to encourage programmers to use the behavior parameterization pattern, because as you've just seen, it makes your code more adaptive to requirement changes. In [chapter 3](#), you'll see that the Java 8 language designers solved this problem by introducing lambda expressions, a more concise way to pass code. Enough suspense; here's a short preview of how lambda expressions can help you in your quest for clean code.

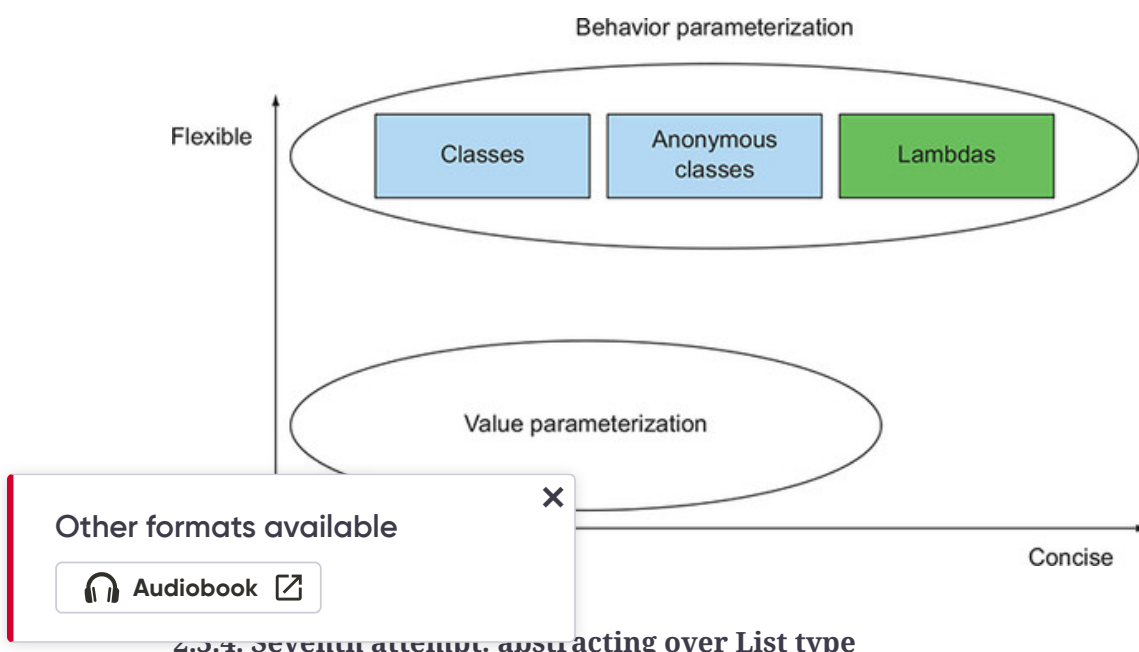
2.3.3. Sixth attempt: using a lambda expression

The previous code can be rewritten as follows in Java 8 using a lambda expression:

```
List<Apple> result =  
    filterApples(inventory, (Apple apple) -> RED.equals(apple.getColor()))
```

You have to admit this code looks a lot cleaner than our previous attempts! It's great because it's starting to look a lot closer to the problem statement. We've now tackled the verbosity issue. [Figure 2.4](#) summarizes our journey so far.

Figure 2.4. Behavior parameterization versus value parameterization



2.3.4. Seventh attempt: abstracting over List type

There's one more step that you can do in your journey toward abstraction. At the moment, the `filterApples` method works only for `Apple`.

But you can also abstract on the `List` type to go beyond the problem domain you're thinking of, as shown:

```
public interface Predicate<T> {
    boolean test(T t);
}

public static <T> List<T> filter(List<T> list, Predicate<T> p) { 1
    List<T> result = new ArrayList<>();
    for(T e: list) {
        if(p.test(e)) {
            result.add(e);
        }
    }
    return result;
}
```

- **1 Introduces a type parameter T**

You can now use the method `filter` with a `List` of bananas, oranges, Integers, or Strings! Here's an example, using lambda expressions:

```
List<Apple> redApples =
    filter(inventory, (Apple apple) -> RED.equals(apple.getColor()));
List<Integer> evenNumbers =
    filter(numbers, (Integer i) -> i % 2 == 0);
```

Isn't it cool? You've managed to find the sweet spot between flexibility and conciseness, which wasn't possible prior to Java 8!

2.4. Real-world examples

You've now seen that behavior parameterization is a useful pattern to easily adapt to changing requirements. This pattern lets you encapsulate a behavior (a piece of code) and parameterize the behavior of methods by passing and using these behaviors you create (for example, different predicates for an `Apple`). We mentioned earlier that this approach is similar to the strategy design pattern. You may have already used this pattern in practice. Many methods in the Java API can be parameterized with different behaviors. These methods are often used together with anonymous

Other formats available



handling.

✕ es, which should solidify the idea of passing comparator, executing a block of code with from a task using `Callable`, and GUI event

2.4.1. Sorting with a Comparator

Sorting a collection is a recurring programming task. For example, say your farmer wants you to sort the inventory of apples based on their weight. Or perhaps he changes his mind and wants you to sort the apples by color. Sound familiar? Yes, you need a way to represent and use different sorting behaviors to easily adapt to changing requirements.

From Java 8, a `List` comes with a `sort` method (you could also use `Collections.sort`). The behavior of `sort` can be parameterized using a `java.util.Comparator` object, which has the following interface:

```
// java.util.Comparator
public interface Comparator<T> {
    int compare(T o1, T o2);
}
```

You can therefore create different behaviors for the `sort` method by creating an ad hoc implementation of `Comparator`. For example, you can use it to sort the inventory by increasing weight using an anonymous class:

```
inventory.sort(new Comparator<Apple>() {
    public int compare(Apple a1, Apple a2) {
        return a1.getWeight().compareTo(a2.getWeight());
    }
});
```

If the farmer changes his mind about how to sort apples, you can create an ad hoc `Comparator` to match the new requirement and pass it to the `sort` method. The internal details of how to sort are abstracted away. With a lambda expression it would look like this:

```
inventory.sort(
    (Apple a1, Apple a2) -> a1.getWeight().compareTo(a2.getWeight()));
```

Again, don't worry about this new syntax for now; the next chapter covers in detail how to write and use lambda expressions.

Other formats available



Audiobook

Code with Runnable

code to be executed concurrently with the
can you tell a thread what block of code it

should run? Several threads may each run different code. What you need is a way to represent a piece of code to be executed later. Until Java 8, only objects could be passed to the `Thread` constructor, so the typical clumsy usage pattern was to pass an anonymous class containing a run

method that returns `void` (no result). Such anonymous classes implement the `Runnable` interface.

In Java, you can use the `Runnable` interface to represent a block of code to be executed; note that the code returns `void` (no result):

```
// java.lang.Runnable
public interface Runnable {
    void run();
}
```

You can use this interface to create threads with your choice of behavior, as follows:

```
Thread t = new Thread(new Runnable() {
    public void run() {
        System.out.println("Hello world");
    }
});
```

But since Java 8 you can use a lambda expression, so the call to `Thread` would look like this:

```
Thread t = new Thread(() -> System.out.println("Hello world"));
```

2.4.3. Returning a result using `Callable`

You may be familiar with the `ExecutorService` abstraction that was introduced in Java 5. The `ExecutorService` interface decouples how tasks are submitted and executed. What's useful in comparison to using threads and `Runnable` is that by using an `ExecutorService` you can send a task to a pool of threads and have its result stored in a `Future`. Don't worry if this is unfamiliar, we will revisit this topic in later chapters when we discuss concurrency in more detail. For now, all you need to know is that the `Callable` interface is used to model a task that returns a result. You can see it as an upgraded `Runnable`:

Other formats available



Audiobook



```
Callable
Callable<V> {
```

You can use it, as follows, by submitting a task to an executor service. Here you return the name of the `Thread` that is responsible for executing

the task:

```
ExecutorService executorService = Executors.newCachedThreadPool();
Future<String> threadName = executorService.submit(new Callable<String>() {
    @Override
    public String call() throws Exception {
        return Thread.currentThread().getName();
    }
});
```

Using a lambda expression, this code simplifies to the following:

```
Future<String> threadName = executorService.submit(
    () -> Thread.currentThread().getName());
```

2.4.4. GUI event handling

A typical pattern in GUI programming is to perform an action in response to a certain event such as clicking or hovering over text. For example, if the user clicks the Send button, you may wish to display a pop up or perhaps log the action in a file. Again, you need a way to cope with changes; you should be able to perform any response. In JavaFX, you can use an `EventHandler` to represent a response to an event by passing it to `setOnAction`:

```
Button button = new Button("Send");
button.setOnAction(new EventHandler<ActionEvent>() {
    public void handle(ActionEvent event) {
        label.setText("Sent!!");
    }
});
```

Here, the behavior of the `setOnAction` method is parameterized with `EventHandler` objects. With a lambda expression it would look like the following:

```
button.setOnAction((ActionEvent event) -> label.setText("Sent!!"));
```

Other formats available



is the ability for a method to *take* multiple different behaviors as parameters and use them internally to *accomplish* different behaviors.

- Behavior parameterization lets you make your code more adaptive to changing requirements and saves on engineering efforts in the future.

- Passing code is a way to give new behaviors as arguments to a method. But it's verbose prior to Java 8. Anonymous classes helped a bit before Java 8 to get rid of the verbosity associated with declaring multiple concrete classes for an interface that are needed only once.
- The Java API contains many methods that can be parameterized with different behaviors, which include sorting, threads, and GUI handling.

Other formats available



Audiobook

